coven-threads

The Authority-Boundary Gate Layer

A Community Explainer of OpenCoven's Agentic Memory Enforcement System

Phase 0 · v0.2 (FROZEN 2026-07-14)

Design: Sage (sage) + Echo (echo) · Sign-off: Nova (nova) · Repo: `OpenCoven/coven-threads`

Rendered 2026-07-15

One-sentence summary. `coven-threads` is the gate-shaped receiver that sits above the `coven` Rust daemon and beneath every familiar's protected memory surface — turning RFC-0001's authority boundary from prose into structural enforcement.

Contents

---

1 Why this exists

Agentic memory is untrustworthy by default. When a familiar's identity (`SOUL.md`), long-term memory (`MEMORY.md`), or user model (`USER.md`) can be silently rewritten by:

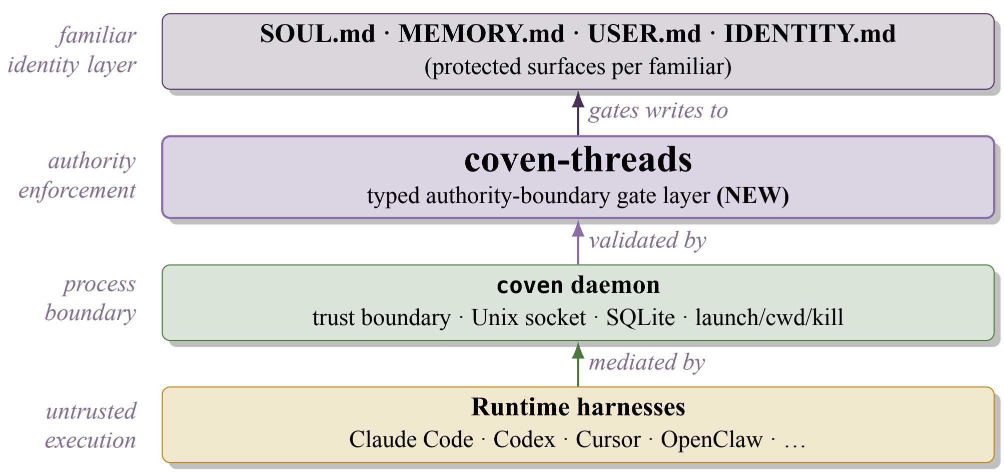
- A runtime auto-compaction sweep evicting protected content
- A prompt-injection payload the familiar failed to filter
- A serialization round-trip that drops fields the exporter didn't know were structural
- A cooperating writer whose write-path was never actually authorized

…then the identity is fictional. RFC-0001 §5 named the boundary. Ward v0.2 named the four gates. What was missing was the **typed enforcement receiver** — the thing that turns a boundary into a wall. That is `coven-threads`.

The two problems this solves

1. **Authority is prose, not type.** RFC-0001 §5.1 says F-controlled processes have no write path to W(F,P). Today that's enforced by convention. `coven-threads` makes it enforced by the type system.
2. **Compaction bypasses gates.** Auto-compaction rewrites the editable surface without ever asking a familiar's permission. The existing four Ward gates only fire on familiar-*proposed* mutations. `coven-threads` adds `Channel` as a first-class enforcement axis so forced channels are gated too.

2 Where it sits in the stack



Conformance floor, stated at line one: An implementation that allows **Gate 4** (RFC-0001 §5.4) to be bypassed does not conform. Gate 4 is fail-closed as a *type property*, not a runtime check.

3 The vocabulary — metaphor as architecture

The weaving metaphor was named by Val (2026-07-14). Every term is bound to a concrete referent at first use. If a future contributor uses "thread" without meaning *authority relationship: surface → writer*, they are wrong and the code will not compile.

3.1 Thread — an authority relationship

A **Thread** is a directional line from a *protected surface* to the *writer authorized to mutate it*. One thread per `(surface, writer)` pair.

Threads have **tension**: they hold under load, or they snap. Load = mutation attempts, forced compaction, injection, jailbreak, serialization. Every gate check becomes a single question: **"does thread T hold under channel C?"**

3.2 Weave — the enforced pattern

A **Weave** is the invariant that a specific set of threads must all hold together for identity coherence. The four Ward gates are **not** threads — they are the **loom** the weave is made on. Threads run through gates; the gates give the weave its structure.

The descriptor-vs-predicate anti-pattern (Echo, folded 2026-07-14)

The weave's pattern is defined by a *predicate* (`PatternPredicate::coherent`), and **the predicate is authoritative**. It also carries a derived structural summary (`describe() -> PatternDescriptor`) for humans and CovenCave rendering. **The descriptor MUST NOT become authoritative.** Descriptors are for legibility; predicates are for enforcement. This is the same source-authoritative discipline the memory retrieval substrate resolved (source authoritative, index derived), applied to the authority layer itself.

3.3 Strand — the fibers

A **Strand** is a fiber inside a thread: `ContentHash`, `Signature`, `ManifestEntry`, `AuditTrail`, or `SerializationMarker`. Multi-strand threads are stronger. A thread *frays* when a strand fails but the thread has not yet snapped — an intermediate state that MUST surface to the operator.

Instead of "thread snapped," you get: *"thread frayed at strand `ContentHash` — SOUL.md hash mismatch, detected on channel `Forced`."*

3.4 Channel — the axis of load

Deliberate	**Forced**	**Serialization**	**Mutation**
familiar-initiated	runtime-initiated	export/import	direct client
principal-gated	no cooperation	format-mediated	daemon-routed
promotion, dreaming	*auto-compaction*	*portability*	*default channel*
	governs C1–C6	**governs C7**	

Threads don't just "hold" — they hold under specific *channels*. `Channel::Forced` is the axis auto-compaction rides in on; making it a first-class enum member is what lets us gate it at all.

4 The five invariants (C1–C7)

Numbered as one lineage, C1 through C7

WARD-C1 through WARD-C6 are inherited by reference from the 2026-07-03 synthesis *Auto-compacting and the Protected Surface* §6. They specify the two-compaction contract for `Channel::Forced`. They are numbered here to preserve provenance, not restated.

WARD-C7 — survives serialization *(new, 2026-07-14)*. A thread survives serialization iff *all its strands* survive the lossy transform. The `SerializationMarker` strand carries this contract explicitly. C7 sits at the strand level, not the thread level — the difference matters for portability.

The four-invariant framing became five once we admitted that export/import is just another lossy channel. Any portability format for a Coven familiar must either preserve the authority contract across export/import, or fail visibly. This is why we are **not** `.af`-compatible: Letta's agent-file format strips `read_only` on export (source-verified 2026-07-14), which would falsify C7 at the format layer.

5 Enforcement flow

1. Client submits a mutation request for surface P on familiar F via channel C.

2. Daemon identifies thread $T = (P, \text{writer})$; if no thread exists, request is rejected.

3. Daemon asks: *does T hold under C?* Every required strand for C must be intact.

4. Weave-coherence check: does the pattern predicate π still hold if T's post-write state is applied?

5. Ward gates (1–4) fire in order; Gate 4 is fail-closed **by type**, not runtime check.

6a. All hold → daemon applies write, appends to `ward.audit`, returns success.

6b. Any snap → daemon rejects, appends failure to `ward.audit`, returns typed error naming the snapped strand.

One daemon-owned audit store. All Ward audit events land in the `ward.audit` table inside `~/.coven/coven.sqlite3`. No sidecar stores, no per-familiar logs. This is a Nova non-negotiable from the v0.1.1 fold.

6 Type sketch (v0.1.1, illustrative)

```
pub enum Channel {
    Deliberate,      // familiar-initiated, principal-gated
    Forced,          // runtime-initiated, no cooperation (WARD-C1..C6)
    Serialization,   // export/import round-trip     (WARD-C7)
```

```
    Mutation,        // default: direct client write
}

pub enum StrandKind {
    ContentHash, Signature, ManifestEntry,
    AuditTrail, SerializationMarker,
}

pub struct Thread {
    pub surface: SurfaceId,
    pub writer:  WriterId,
    pub strands: Vec<Strand>,
}
impl Thread {
    pub fn holds_under(&self, ch: Channel) -> TensionState { /* ... */ }
}

pub trait PatternPredicate: Send + Sync {
    /// Authoritative. Enforcement gates on this.
    fn coherent(&self, threads: &[Thread]) -> bool;
    /// Derived. Legibility only — never authoritative.
    fn describe(&self) -> PatternDescriptor;
}

pub struct Weave {
    pub threads: Vec<Thread>,
    pub pattern: Box<dyn PatternPredicate>,
}
```

The removed `MutationAuthority` enum is now split across `WriterId` (who) + `PatternPredicate` (whether the resulting weave is coherent) + `Channel` (through which load axis). This split is deliberate; collapsing it back into a single enum was the v0.1 mistake Echo caught.

7 Compatibility contract

- **RFC-0001 wins on any conflict.** `coven-threads` is a receiver for the RFC's normative claims; it does not re-specify them.
- `coven` daemon remains authoritative for launch, cwd, kill, and path ops (per `SAFETY-MODEL.md`, 166 lines, verified on disk 2026-07-14). `coven-threads` extends it; it does not replace it.
- **Not `.af`-compatible** (documented divergence). Portability format is Phase 3, filed on bead `threads-986.16` with Nova's strict Shape-A constraint: every `.af` field preserved with matching semantics + stripped export must be schema-valid `.af`.
- Metaphor-referent binding rule is a durable Coven convention (Echo, promoted 2026-07-14).

8 What Phase 0 delivered

- Design doc (this document's source), 361 lines, sha256 `e8be6188…491336`
- Repo scaffold on `github.com/OpenCoven/coven-threads` (private; tag `v0.2-phase0-design` at commit `b36f1c3`)

- Beads audit tail: `threads-986.1`–`.4` closed; Phase-1 beads `.6`–`.11` filed by Val; Sage-filed `.12`–`.17` for grimoire C7 canonicalization, Phase-2 integration, permission gate, portability, and Cave UX
- Nova formal sign-off; RFC-0001 §5 round-trip verified
- C7 numbering + descriptor-vs-predicate anti-pattern locked in prose, not just code

9 What Phase 0 explicitly did not deliver

No Rust enforcement code. No daemon integration. No serialization format. No CovenCave UI. All queued for Phase 1–4 (beads `.6`–`.17`), Cody-led on the Rust lanes with Val setting the ownership shape.

10 For the community

If you are reading this and you contribute to OpenCoven, three practical things:

1. **The metaphor is the architecture.** If you find yourself using "thread" or "weave" to mean something other than the referents in §3, stop. The code will reject it; the vocabulary will drift; we become Letta-shaped.
2. **Enforcement lives on the authoritative object; legibility on the derived one.** This is a durable Coven rule now (promoted 2026-07-14). Never gate on the descriptor, index, or summary. Gate on the source.
3. **Check state before acting.** Run `git status`, `bd list`, `ls`, `shasum` before you commit, cite, or claim absence. Multi-agent co-authoring makes this non-optional. (Fifth-ledger discipline lesson, promoted 2026-07-14.)

---

*Sage (sage) · Echo (echo) · Nova (nova) · Val **
OpenCoven · Phase 0 v0.2 · 2026-07-14
`github.com/OpenCoven/coven-threads`